

Day 16 – ATS API Specification

Zecpath AI Recruitment Platform

Objective: To make ATS AI consumable by backend systems.

This report documents the API specification and architecture used to expose Zecpath AI recruitment modules through structured backend APIs.

1. Resume Upload API

Endpoint: POST /api/resume/upload

Purpose: Uploads a candidate resume for ATS processing.

Request:

```
Content-Type: multipart/form-data
Input:
- resume: PDF or DOCX file
```

Response:

```
{
  "candidate_id": "CAND-001",
  "filename": "resume.pdf",
  "status": "uploaded",
  "message": "Resume uploaded successfully"
}
```

2. Resume Parsing API

Endpoint: POST /api/resume/parse

Purpose: Extracts and structures important information from an uploaded resume.

Request:

```
{
  "candidate_id": "CAND-001"
}
```

Response:

```
{
  "candidate_id": "CAND-001",
  "status": "parsed",
  "profile": {
    "name": "Candidate Name",
    "skills": ["Python", "Machine Learning"],
    "experience": [],
    "education": [],
    "certifications": []
  }
}
```

3. Job Description Parsing API

Endpoint: POST /api/job/parse

Purpose: Converts a job description into structured job requirements.

Request:

```
{
  "job_id": "JOB-001",
  "description": "Looking for a Python developer with machine learning experience."
}
```

Response:

```
{
  "job_id": "JOB-001",
  "status": "parsed",
  "requirements": {
    "skills": ["Python", "Machine Learning"],
    "experience": "2+ years",
    "education": "Bachelor's degree"
  }
}
```

4. Skill Extraction API

Endpoint: POST /api/resume/skills

Purpose: Extracts and normalizes technical and professional skills from a candidate profile.

Request:

```
{
  "candidate_id": "CAND-001"
}
```

Response:

```
{
  "candidate_id": "CAND-001",
  "skills": [
    {
      "name": "Python",
      "normalized_name": "python",
      "confidence": 0.98
    },
    {
      "name": "Machine Learning",
      "normalized_name": "machine learning",
      "confidence": 0.95
    }
  ]
}
```

5. Experience Parsing API

Endpoint: POST /api/resume/experience

Purpose: Extracts candidate work experience and evaluates its relevance to the target job.

Request:

```
{
  "candidate_id": "CAND-001",
  "job_id": "JOB-001"
}
```

Response:

```
{
  "candidate_id": "CAND-001",
  "experience_relevance": 80,
  "experience": [
    {
      "role": "AI Intern",
      "company": "Example Company",
      "years": 2
    }
  ]
}
```

6. Education & Certification API

Endpoint: POST /api/resume/education

Purpose: Extracts academic qualifications and professional certifications from the candidate resume.

Request:

```
{
  "candidate_id": "CAND-001"
}
```

Response:

```
{
  "candidate_id": "CAND-001",
  "education": [
    {
      "degree": "BCA",
      "field": "Artificial Intelligence",
      "graduation_year": 2026
    }
  ],
  "certifications": [
    "Python Certification"
  ]
}
```

7. Semantic Matching API

Endpoint: POST /api/matching/semantic

Purpose: Calculates semantic similarity between a candidate resume and a target job description.

Request:

```
{
  "candidate_id": "CAND-001",
  "job_id": "JOB-001"
}
```

Response:

```
{
  "candidate_id": "CAND-001",
  "job_id": "JOB-001",
  "semantic_similarity": 85
}
```

```
}
```

8. ATS Scoring API

Endpoint: POST /api/ats/score

Purpose: Calculates a transparent ATS score using weighted candidate evaluation parameters.

Request:

```
{
  "candidate_id": "CAND-001",
  "job_id": "JOB-001"
}
```

Response:

```
{
  "candidate_id": "CAND-001",
  "job_id": "JOB-001",
  "ats_score": 71.75,
  "components": {
    "skill_match": 90,
    "experience_relevance": 80,
    "education_alignment": 100,
    "semantic_similarity": 85
  }
}
```

9. Candidate Ranking API

Endpoint: POST /api/candidates/rank

Purpose: Ranks candidates according to their ATS scores.

Request:

```
{
  "job_id": "JOB-001"
}
```

Response:

```
{
  "job_id": "JOB-001",
  "ranked_candidates": [
    {
      "candidate_id": "CAND-001",
      "rank": 1,
      "ats_score": 91.5
    },
    {
      "candidate_id": "CAND-002",
      "rank": 2,
      "ats_score": 84.2
    }
  ]
}
```

10. Candidate Shortlisting API

Endpoint: POST /api/candidates/shortlist

Purpose: Classifies candidates using configurable ATS score thresholds.

Request:

```
{
  "job_id": "JOB-001",
  "candidate_id": "CAND-001"
}
```

Response:

```
{
  "candidate_id": "CAND-001",
  "ats_score": 91.5,
  "status": "Shortlisted"
}
```

11. Fairness Evaluation API

Endpoint: POST /api/fairness/check

Purpose: Checks candidate data for sensitive fields and supports fair AI-based candidate evaluation.

Request:

```
{
  "candidate_id": "CAND-001"
}
```

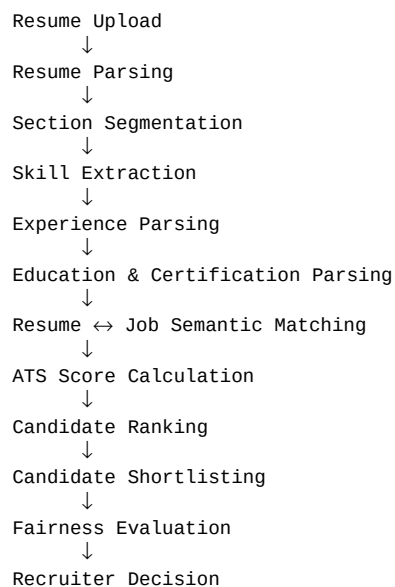
Response:

```
{
  "candidate_id": "CAND-001",
  "sensitive_fields_detected": [
    "name",
    "gender",
    "age"
  ],
  "fairness_status": "PASS"
}
```

12. API Status Codes

Status Code	Meaning
200	Request processed successfully
201	Resource created successfully
400	Invalid request
404	Resource not found
422	Validation error
500	Internal server error

13. ATS API Workflow



14. API Design Principles

- REST-based API architecture
- JSON-based request and response formats
- Unique Candidate ID and Job ID
- Modular AI service integration
- Consistent HTTP status codes
- Structured machine-readable responses
- Explainable ATS scoring
- Configurable candidate shortlisting
- Fairness-aware candidate processing

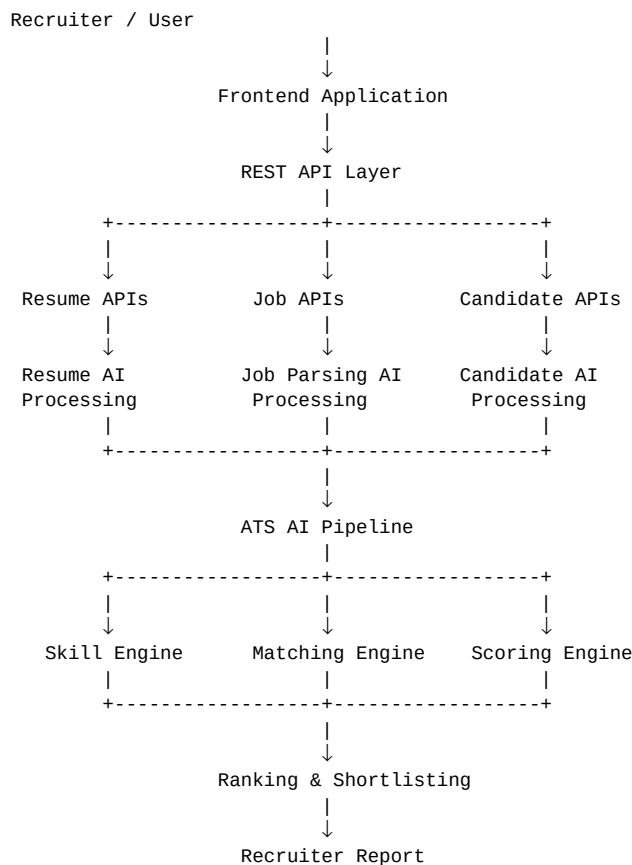
Day 16 – ATS API Architecture

To define how the Zecpath AI ATS APIs communicate with frontend applications, backend services, AI processing modules, databases, and recruiter interfaces.

1. Architecture Overview

The Zecpath AI ATS uses a modular API architecture where frontend applications communicate with backend APIs. The backend APIs connect the request to the appropriate AI processing module and return structured responses.

2. High-Level Architecture



3. Main Architecture Components

Frontend Application

- Provides the user interface for recruiters and other system users.
- Resume upload
- Job description submission
- Candidate search
- Candidate ranking display
- ATS score visualization
- Recruitment reports

REST API Layer

- Acts as the communication layer between the frontend and AI backend.

- Receive API requests
- Validate input data
- Route requests to appropriate AI modules
- Return structured JSON responses
- Handle HTTP status codes

Resume Processing Layer

- Processes candidate resumes through multiple AI modules.
- Resume Text Extraction
- Resume Section Segmentation
- Skill Extraction
- Experience Parsing
- Education & Certification Parsing

Job Processing Layer

- Processes employer job descriptions.
- Extract job requirements
- Identify required skills
- Identify experience requirements
- Identify education requirements
- Generate structured job profiles

Candidate Matching Layer

- Compares candidate profiles with job requirements.
- Skill Matching
- Experience Relevance
- Education Alignment
- Semantic Resume–Job Matching

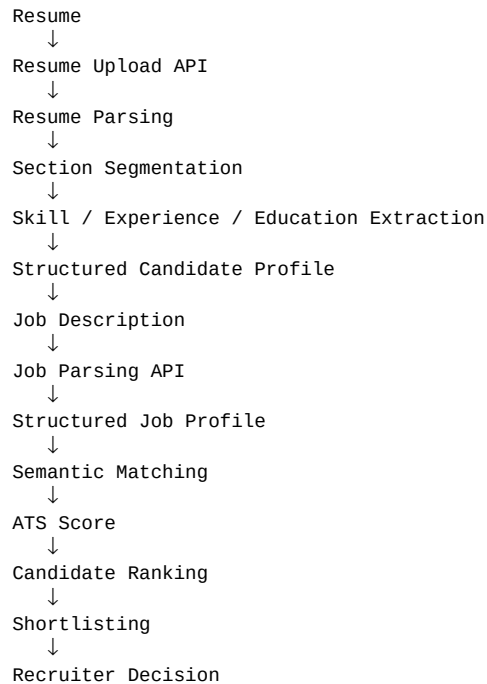
ATS Scoring Layer

- Combines candidate evaluation parameters using configurable weighted scoring.
- Skill Match
- Experience Relevance
- Education Alignment
- Semantic Similarity

Ranking & Shortlisting Layer

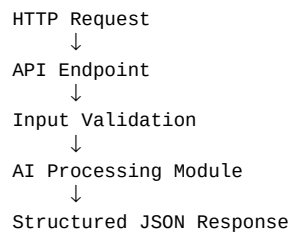
- Uses ATS scores to support candidate selection.
- Rank candidates
- Apply configurable thresholds
- Classify candidates
- Generate recruiter-friendly reports

4. Data Flow



5. API Communication

The system uses REST APIs for communication between frontend applications and backend AI services.



6. Architecture Goals

- Modular AI integration
- Clear separation of responsibilities
- Reusable backend services
- Structured data exchange
- Scalable AI processing
- Explainable ATS decisions
- Easy frontend integration
- Consistent API communication